PROYECTO NANOFILES

Curso 25/26; Convocatoria Enero-Enero
Redes de Comunicaciones
Subgrupo 9.1
Profesor:

Pedro Blaya Pascual (48853636A)
Daniel Pozas Ruiz

ÍNDICE

ÍNDICE **1**
1. INTRODUCCIÓN **2**
2. PROTOCOLOS DISEÑADOS **2**
2.1. Directorio **2**
2.1.1 Formato de los mensajes y ejemplos 2
2..1.1.1 Tipos y descripción de los mensajes 2
403 ..... **4**
Analisis, diseño y eficiencia 4
Listado de código 5
404 ..... **8**
Analisis, diseño y eficiencia 8
Listado de código 9
405 ..... **14**
Analisis, diseño y eficiencia 14
Listado de código 15
407 ..... **22**
Analisis, diseño y eficiencia 22
Listado de código 23
408 ..... **28**
Analisis, diseño y eficiencia 28
Listado de código 29
409 ..... **35**
Analisis, diseño y eficiencia 35
Listado de código 36
411 ..... **40**
Analisis, diseño y eficiencia 40
Listado de código 41
412 ..... **44**
Analisis, diseño y eficiencia 44
Listado de código 45
USO DE IA **50**
CONCLUSIONES **53**

1. INTRODUCCIÓN

Este es un primer boceto de nuestra memoria para el proyecto de NanoFiles. En este documento se expone cómo hemos diseñado los distintos mensajes de comunicación, tanto peer-to-peer como un peer con el directorio. También se incluye una representación gráfica de los autómatas con estados finitos. Toda esta implementación de mensajes y autómatas vendrá acompañada con el código correspondiente a su implementación.

En cuanto a las mejoras, hemos considerado la implementación de todas las posibles.

2. PROTOCOLOS DISEÑADOS

2.1. Directorio.

2.1.1 Formato de los mensajes y ejemplos.

Para definir el protocolo de comunicación con el Directorio, vamos a utilizar mensajes textuales con formato “campo:valor”. El valor que tome el campo “operation” (código de operación) indicará el tipo de mensaje y por tanto su formato (que campos vienen a continuación).

2..1.1.1 Tipos y descripción de los mensajes

Mensaje: ping
Sentido de la comunicación: Cliente -> Directorio
Descripción: contactar con el servidor de directorio para comprobar que está a la escucha y que utiliza un protocolo compatible con el del peer.
Ejemplo:
operation:pin\n
protocolid:12345678A\n

Mensaje: ping_ok
Sentido de la comunicación: Cliente <- Directorio
Descripción: indicar que el protocolo que acompaña al mensaje ping del cliente es igual al que espera el servidor(Directorio).
Ejemplo:
operation:pin_ok\n

Mensaje:ping_error
Sentido de la comunicación: Cliente <- Directorio
Descripción: indicar que el protocolo que acompaña al mensaje ping del cliente no es igual al que espera el servidor(Directorio)
Ejemplo:
operation:ping_error\n

Mensaje: dirfiles
Sentido de la comunicación: Cliente -> Directorio
Descripción: pide la lista de ficheros almacenados en el directorio.
Ejemplo:
operation:dirfiles\n

Mensaje: dirfiles_ok
Sentido de la comunicación: Cliente <- Directorio
Descripción: listado de ficheros donde para cada fichero se indica su nombre, tamaño y hash. Se tiene en cuenta que el listado puede acomodarse en más de un datagrama.
Ejemplo:
operation:dirfiles_ok\n
isLast:boolean\n
file:name,size,hash\n
file:name,size,hash\n
.
.
.
file:name,size,hash\n

Mensaje: serve
Sentido de la comunicación: Cliente -> Directorio
Descripción: pide al directorio registrarse como un servidor de ficheros que escuche conexiones en un puerto TCP, y registrar dicha dirección de escucha (IP y puerto) en el directorio, asociándola al nickname del peer.
Ejemplo:
operation:server\n
serve:name,IP,puerto\n

Mensaje: serve_ok
Sentido de la comunicación: Cliente <- Directorio
Descripción: indica que se ha podido registrar como servidor ese peer
Ejemplo:
operation:serve_ok\n

Mensaje: serve_error
Sentido de la comunicación: Cliente <- Directorio
Descripción: indicar que no se ha podido registrar como servidor ese peer porque el nombre ya está registrado.
Ejemplo:
operation:serve_error\n

Mensaje: peers
Sentido de la comunicación: Cliente -> Directorio
Descripción: pide el censo de peers registrados como servidores en el directorio.
Ejemplo:
operation:peers\n

Mensaje: peers_ok
Sentido de la comunicación: Cliente <- Directorio
Descripción: muestra el censo de peers registrados como servidores en el directorio. El listado indica para cada par su nickname, IP y puerto TCP de escucha.
Ejemplo:
operation:serve_ok\n
serve:name,IP,puerto\n
serve:name,IP,puerto\n
.
.
.
serve:name,IP,puerto\n

Mensaje: quit
Sentido de la comunicación: Cliente -> Directorio
Descripción: cierra la aplicación. Si el servidor estaba activo, se detiene y se solicita la baja al directorio.
Ejemplo:
operation:quit\n

403.

Analisis, diseño y eficiencia.

Para la resolución de este problema he declarado la clase GrafoLista para modular el código. Los atributos de esta clase son la lista de adyacencia implementada mediante la idea de listas ligadas(debido al extremado número de nodos que se pueden llegar a obtener, 20000), el vector visitados necesario para la bpp o la bpa y el número de nodos que tendrá el grafo. La idea que he seguido para solucionar este problema ha sido realizar una búsqueda primero en profundidad(bpp) sobre el grafo. Como queremos ver el recorrido que

realiza el personaje hasta llegar a la salida, declare un vector que sirviese como apoyo a la función bpp_lab(función que realiza una bpp normal) para guardar el recorrido que se debe realizar para llegar al final. Los nodos que se van insertando en este vector son los nodos recorridos por la bpp hasta llegar al nodo querido. Para distinguir si se puede llegar al final del laberinto o no, es ver si el nodo principal(1) y el último(n) están en el mismo grafo conexo. Para ello hacemos que bpp_lab devuelve un booleano para comprobar si desde 1 hemos llegado a 4. Gracias a ese booleano, en busquedaLaberinto, devolvemos un vector vacío si no se ha encontrado o uno con el recorrido sí sí se ha encontrado. Finalmente en el main imprimimos el texto correspondiente al caso que nos proporcione ese vector de retorno de la función busquedaLaberinto.

En cuanto a la eficiencia, como es una búsqueda primero en profundidad tendremos O(n+a), siendo n el número de nodos y a la longitud media de las listas. Sin embargo; esta eficiencia se ve afectada por la función leeGrafo, ya que las inserciones en las listas se realizan al final(O(a), donde a es la longitud de la lista).

Listado de código.

```
#include <stdlib.h>
#include <string.h>
#include <vector>
#include <iostream>
#include <sstream>
using namespace std;

#define MAX_NODOS 20000
class GrafoLista
{
    private:
    struct NodoPtr
    {
        int w;
        NodoPtr* sig;
        NodoPtr():w(-1),sig(nullptr){};
        NodoPtr( int w ): w(w), sig(nullptr){};
    };
    int num_nodos;
    vector<bool> visitados;
    vector<NodoPtr*> listaAdyacencia;
```

```
    public:
    GrafoLista(): num_nodos(0){};
    GrafoLista(int num_nodos)
    {
        this->num_nodos = num_nodos;
        listaAdyacencia.assign(num_nodos, nullptr);
        visitados.assign(num_nodos, false);
    }
    static GrafoLista leeGrafo ()
    {
        int num_nodos;
        cin >> num_nodos;
        cin.ignore();
        if (num_nodos < 0 || num_nodos > MAX_NODOS) {
            cerr << "Numero de nodos (" << num_nodos << ") no
valido\n";
            exit(0);
        }
        GrafoLista lectura = GrafoLista(num_nodos);
        string nodos_adyacentes;
        int w;
        for (int v= 0; v < num_nodos; v++) {
            getline(cin, nodos_adyacentes);
            istringstream ss( nodos_adyacentes );
            while( ss >> w)
            {
                w--;
                if(w >= 0 && w < num_nodos )
                {
                    NodoPtr* insertar = new NodoPtr(w);
                    if ( lectura.listaAdyacencia[v] == nullptr)
                    {
                        lectura.listaAdyacencia[v] = insertar;
                    }
                    else
                    {

                        NodoPtr* aux = lectura.listaAdyacencia[v];
                        while ( aux -> sig != nullptr )
                        {
                            aux = aux -> sig;
                        }
                        aux -> sig = insertar;
```

```
                    }
                }
            }
        }
        return lectura;
    }
    bool bpp_lab(int v, vector<int>& recorrido)
    {
        recorrido.push_back(v+1);
        visitados[v] = true;
        if ( v == num_nodos - 1 )
        {
            return true;
        }
        NodoPtr* aux = listaAdyacencia[v];
        while ( aux != nullptr )
        {
            int w = aux -> w;
            if( !visitados[w])
            {
                bool encontrado = bpp_lab(w, recorrido);
                if( encontrado )
                {
                    return encontrado;
                }
                recorrido.push_back(v + 1);
            }
            aux = aux -> sig;
        }
    return false;
}

    vector<int> busquedaLaberinto()
    {
        vector<int> recorrido;
        bool resultado = bpp_lab(0, recorrido);
        if ( resultado == false )
        {
            recorrido.clear();
        }
        return recorrido;

    }
```

```
};
int main (void)
{
    int num_casos;
    cin >> num_casos;
    cin.ignore();
    for (int i= 0; i < num_casos; i++)
    {
        GrafoLista laberinto = GrafoLista::leeGrafo();
        vector<int> recorrido = laberinto.busquedaLaberinto();
        cout << "Caso " << (i+1) << endl;
        if( recorrido.empty() )
        {
            cout << "INFINITO" << endl;
        }
        else
        {
            int longitud_camino = recorrido.size();
            cout << longitud_camino << endl;
            for ( int i = 0; i < longitud_camino; i++)
            {
                cout << recorrido[i] << endl;
            }
        }
    }
  return 0;
}
```

404.

Analisis, diseño y eficiencia.

Para la resolución de este problema he declarado la clase GrafoLista para modular el código. Los atributos de esta clase son la lista de adyacencia implementada mediante la idea de listas ligadas, el vector visitados necesario para la bpp o la bpa(aunque en este caso no se usa), el número de nodos que tendrá el grafo y los valores de los nodos para los que queremos comprobar el camino. La idea que he seguido para solucionar este problema ha sido aplicar el algoritmo de Dijkstra sobre el grafo que nos proporcionan, pero con la peculiaridad de que se toma como inicio el nodo A y se detiene cuando se ha llegado al nodo B. Así podemos aplicar este algoritmo primero sobre los nodos A y C y

luego C y B.Por lo tanto, la solución será sumar el valor que ha almacenado este algoritmo para llegar a C en la primera llamada más el valor almacenado para llegar a B en la segunda llamada. Como podría ser que no existiese un camino de A a C o de C a B, tratamos este caso considerando que este algoritmo puede devolver la constante INFINITO definida para representar que no hay arista que una dos nodos. Resumiendo, este algoritmo es el mismo que aparece en las transparencias pero con las peculiaridades ya descritas. Una vez hecho las distinciones sobre qué nos puede devolver la función que reproduce el algoritmo de Dijkstra, el main solo consta de diferenciar estos casos e imprimir el texto correspondiente al caso obtenido.

En cuanto a la eficiencia, el algoritmo de Dijkstra tiene normalmente O(n*a*log(n)), siendo n el número de nodos y a la longitud media de las listas. Sin embargo; esta eficiencia se ve afectada por la función leeGrafo, ya que las inserciones en las listas se realizan al final(O(a), donde a es la longitud de la lista).

Listado de código.

```
#include <stdlib.h>
#include <string.h>
#include <queue>
#include <vector>
#include <iostream>
#include <sstream>
using namespace std;

#define MAX_NODOS 20000
#define INFINITO  2147483000
class GrafoLista
{
    private:
    struct NodoPtr
    {
        int w;
        int coste;
        NodoPtr* sig;
        NodoPtr():w(-1),sig(nullptr){};
        NodoPtr( int w, int coste ): w(w), coste(coste),
sig(nullptr){};;

    };
    int num_nodos;
    int A;
```

```
    int B;
    int C;
    vector<bool> visitados;
    vector<NodoPtr*> listaAdyacencia;
    bool isMiembro( int v, int w)
    {
        NodoPtr* aux = listaAdyacencia[v];
        while ( aux != nullptr && aux ->w != w)
        {
            aux = aux -> sig;
        }
        if ( aux != nullptr)
        {
            return true;
        }
        return false;
    }
    int valor(int u, int v)
    {
        NodoPtr* aux = listaAdyacencia[u];
        while ( aux != nullptr && aux->w != v )
        {
            aux = aux -> sig;
        }
        return aux->coste;
    }
    int elegido( vector<bool>& nodos_escogidos, vector<int>&
coste_caminos)
    {
        int minimo = INFINITO;
        int indice = -1;
        for ( int v = 0; v < num_nodos; v++)
        {
            if ( !nodos_escogidos[v] && coste_caminos[v] < minimo)
            {
                minimo = coste_caminos[v];
                indice = v;
            }
        }
        return indice;
    }

    public:
```

```
    GrafoLista(): num_nodos(0){};
    GrafoLista(int num_nodos, int A, int B, int C)
    {
        this->num_nodos = num_nodos;
        this->A = A;
        this->B = B;
        this->C = C;
        listaAdyacencia.assign(num_nodos, nullptr);
        visitados.assign(num_nodos, false);
    }
    void insertarnodo(int v, int w, int coste)
    {
        NodoPtr* insertar = new NodoPtr(w, coste);
        if ( this -> listaAdyacencia[v] == nullptr)
        {
            this -> listaAdyacencia[v] = insertar;
        }
        else
        {

            NodoPtr* aux = this -> listaAdyacencia[v];
            while ( aux -> sig != nullptr )
            {
                aux = aux -> sig;
            }
            aux -> sig = insertar;
        }
    }
     static GrafoLista leeGrafo ()
    {
        //PARAMETROS

        int num_nodos;
        int aristas;
        int A,B,C;
        cin >> num_nodos;
        cin >> aristas;
        cin >> A >> B >> C;

        //ERROR TAMAÑO LISTA

        if (num_nodos < 0 || num_nodos > MAX_NODOS) {
```

```
            cerr << "Numero de nodos (" << num_nodos << ") no
valido\n";
            exit(0);
        }

        GrafoLista lectura(num_nodos, A - 1, B - 1, C - 1);

        //LECTURA ARISTAS
        for (int i= 0; i < aristas ; i++)
        {
            int v, w, coste;
            cin >> v >> w >> coste;
            cin.ignore();
            v--;
            w--; //AJUSTAR A INDICES ARRAY
            //FALTA INSERTAR INSERTAR W -> V PQ ES NO DIRIGIDO

            //INSERCION NODOS
            if(w >= 0 && w < num_nodos && v >= 0 && v < num_nodos )
            {
                lectura.insertarnodo(v,w,coste);
                lectura.insertarnodo(w,v,coste);
            }
        }
        return lectura;
    }
    int getA()
    {
        return A;
    }
    int getB()
    {
        return B;
    }
    int getC()
    {
        return C;
    }
    int algoritmo_Dijkstra_coste_camino(int A, int B)
    {
        vector<int> nodos_paso(num_nodos, A);
        vector<int> coste_caminos(num_nodos, INFINITO);
        vector<bool> nodos_escogidos(num_nodos, false);
```

```
        //INICIALIZACION
        for(int v = 0; v < num_nodos ; v ++)
        {
            if ( v != A  && isMiembro(A,v) )
            {
                coste_caminos[v]= this->valor(A,v);
            }
            if ( v == A )
            {
                nodos_escogidos[v] = true;
                coste_caminos[v] = 0;
                nodos_paso[v] = -1;
            }
        }
        //CUERPO
        bool encontrado_B = false;
        while ( !encontrado_B )
        {
            int v = elegido(nodos_escogidos, coste_caminos);
            if ( v == -1 )
            {
                break;
            }
            nodos_escogidos[v] = true;
            NodoPtr* aux = listaAdyacencia[v];
            while( aux != nullptr )
            {
                int w = aux ->w;
                int coste_v_w = coste_caminos[v] + aux ->coste;
                if ( !nodos_escogidos[w] && (coste_v_w <
coste_caminos[w]))
                {
                    coste_caminos[w] = coste_v_w;
                    nodos_paso[w] = v;
                }
                aux = aux -> sig;
            }
            if ( v == B )
            {
                encontrado_B = true;
            }
        }
        return coste_caminos[B];
```

```
    }
};
int main (void)
{
    int num_casos;
    cin >> num_casos;
    cin.ignore();
    for (int i= 0; i < num_casos; i++)
    {
        GrafoLista caminos = GrafoLista::leeGrafo();
        int camino_A_C =
caminos.algoritmo_Dijkstra_coste_camino(caminos.getA(),
caminos.getC());
        int camino_C_B =
caminos.algoritmo_Dijkstra_coste_camino(caminos.getC(),
caminos.getB());
        if( camino_A_C == INFINITO || camino_C_B == INFINITO )
        {
            cout << "IMPOSIBLE" << endl;
        }
        else
        {
            cout << camino_A_C + camino_C_B << endl;
        }
    }
  return 0;
}
```

405.

Analisis, diseño y eficiencia.

Para la resolución de este problema he declarado la clase GrafoLista para modular el código. Los atributos de esta clase son la lista de adyacencia implementada mediante la idea de listas ligadas, el vector visitados necesario para la bpp o la bpa(aunque en este caso no se usa) y el número de nodos que tendrá el grafo. La idea que he seguido para solucionar este problema ha sido aplicar el algoritmo de Dijkstra sobre el grafo que nos proporcionan, pero con la peculiaridad de que se toma como inicio el nodo A y que va actualizando los valores de los costes de las aristas. Como los encuentros para los cotilleos se realizan en el instante x y si ese instante ya ha pasado y no se ha transmitido el cotilleo hay que esperar al día siguiente, entonces

cuando se elige un nodo para inspeccionar sus adyacentes primero se actualizan los valores que sean menores al instante que tenga marcado el nodo elegido. Después de esa actualización ya se puede recorrer sus adyacentes para ver si hay que actualizar el tiempo mínimo en el que se van a enterar del cotilleo, si se diera el caso que si hubiese que actualizarla, no se realiza como en el algoritmo Dijkstra normal, sino que se reemplaza el valor almacenado por el que contiene el adyacente directamente. También he usado una estructura auxiliar (ID), para poder llevar a cabo las actualizaciones anteriores, porque aparte del nodo a explorar necesitamos, también, el valor por el cual se ha elegido, ya que pueden haber diferentes instantes en los que se pueden producir los encuentros. Por último, la función encargada de realizar el algoritmo Dijkstra devuelve el mayor valor del vector que almacena los instantes mínimos en los que se entera cada persona. Este valor puede ser -1, lo que nos dice que habrá personas que no se habrán enterado del cotilleo, u otro diferente a -1 que será el instante en el que la última persona se ha enterado del cotilleo. En el main se tiene en cuenta los posibles resultados y se imprime el texto correspondiente al caso obtenido.

En cuanto a la eficiencia, el algoritmo de Dijkstra tiene normalmente O(n*a*log(n)), siendo n el número de nodos y a la longitud media de las listas. Sin embargo; esta eficiencia se ve afectada por la función leeGrafo, ya que las inserciones en las listas se realizan al final(O(a), donde a es la longitud de la lista).

Listado de código.

```
#include <stdlib.h>
#include <string.h>
#include <vector>
#include <list>
#include <iostream>
#include <sstream>
using namespace std;

#define MAX_NODOS 20000
#define INFINITO  2147483000
class GrafoLista
{
    private:
    struct NodoPtr
    {
        int w;
```

```
        vector<int> datos;
        NodoPtr* sig;
        NodoPtr():w(-1),sig(nullptr){};
        NodoPtr( int w ): w(w), datos(), sig(nullptr){};;

    };
    struct ID
    {
        int v;
        int coste;
        ID(int v, int coste): v(v), coste(coste){};
    };
    int num_nodos;
    vector<bool> visitados;
    vector<NodoPtr*> listaAdyacencia;

    NodoPtr* posicion_nodo_v_w ( int v, int w)
    {
        NodoPtr* aux = listaAdyacencia[v];
        while ( aux != nullptr && aux->w != w)
        {
            aux = aux->sig;
        }
        return aux;
    }

    vector<int> valor(int v, int w)
    {
        NodoPtr* aux = posicion_nodo_v_w(v, w);
        if ( aux == nullptr)
        {
            return vector<int>();
        }
        return aux->datos;
    }

    int menor_valor_vector( vector<int>& lista_costes)
    {
        int minimo = INFINITO;
        for ( size_t i = 0; i < lista_costes.size(); i++)
        {
            if ( lista_costes[i] < minimo)
            {
```

```
                minimo = lista_costes[i];
            }
        }
        return minimo;
    }

    int mayor_valor_vector( vector<int>& lista_costes ) // devuelve -1
si hay personas que no saben el cotilleo y sino devuelve el tiempo del
que mas tarda en enterarse
    {
        int maximo = -1;
        size_t indice = 0;
        bool conexo = true;
        while ( indice < lista_costes.size() && conexo == true)
        {
            if ( lista_costes[indice] != INFINITO )
            {
                if( lista_costes[indice] > maximo)
                {
                    maximo = lista_costes[indice];
                }
            }
            else
            {
                conexo = false;
            }
            indice++;
        }
        if ( conexo )
        {
            return maximo;
        }
        return -1;
    }

    ID elegido( vector<bool>& nodos_escogidos, vector<int>&
coste_caminos)
    {
        int minimo = INFINITO;
        int indice = -1;
        for ( int v = 0; v < num_nodos; v++)
        {
            if ( !nodos_escogidos[v] && coste_caminos[v] < minimo)
```

```
            {
                minimo = coste_caminos[v];
                indice = v;
            }
        }
        return ID(indice, minimo);
    }

    void actualizar( ID id, vector<int>& coste_caminos, vector<int>&
nodos_paso )
    {
        NodoPtr* aux_v_w = listaAdyacencia[id.v];

        while ( aux_v_w != nullptr )
        {
            //valor del adyacente a v
            int w = aux_v_w ->w;

            //nodo (w,v)
            NodoPtr* aux_w_v = posicion_nodo_v_w( w, id.v);

            //tratamiento del vector de (v,w)
            for ( size_t i = 0; i < aux_v_w -> datos.size(); i ++)
            {
                while ( aux_v_w -> datos[i] < id.coste )
                {
                    aux_v_w -> datos[i] += 100;
                }

                //guardar actualizacion en el nodo (w,v)
                aux_w_v -> datos[i] = aux_v_w -> datos[i];
            }

            //actualizacion de coste_caminos
            int menor_valor_v_w = menor_valor_vector( aux_v_w -> datos
);
            if ( menor_valor_v_w < coste_caminos[w])
            {
                coste_caminos[w] = menor_valor_v_w;
                nodos_paso[w] = id.v;
            }

            aux_v_w = aux_v_w ->sig ;
```

```
        }
    }
    void insertarnodo(int v, int w, int veces)
    {
        NodoPtr* insertar_w = new NodoPtr(w);
        NodoPtr* insertar_v = new NodoPtr(v);
        for( int i = 0 ; i < veces; i++)
        {
            int valor;
            cin >> valor;
            insertar_w -> datos.push_back(valor);
        }
        cin.ignore();
        //insertar (v,w)
        if ( this -> listaAdyacencia[v] == nullptr)
        {
            this -> listaAdyacencia[v] = insertar_w;
        }
        else
        {

            NodoPtr* aux = this -> listaAdyacencia[v];
            while ( aux -> sig != nullptr )
            {
                aux = aux -> sig;
            }
            aux -> sig = insertar_w;
        }
        //insertar (w,v)
        insertar_v->datos = insertar_w->datos;
        if ( this -> listaAdyacencia[w] == nullptr)
        {
            this -> listaAdyacencia[w] = insertar_v;
        }
        else
        {

            NodoPtr* aux = this -> listaAdyacencia[w];
            while ( aux -> sig != nullptr )
            {
                aux = aux -> sig;
            }
            aux -> sig = insertar_v;
```

```
        }
    }
    public:
    GrafoLista(): num_nodos(0){};

    GrafoLista(int num_nodos)
    {
        this->num_nodos = num_nodos;
        listaAdyacencia.assign(num_nodos, nullptr);
        visitados.assign(num_nodos, false);
    }

     static GrafoLista leeGrafo ()
    {
        //PARAMETROS

        int num_nodos;
        int aristas;
        cin >> num_nodos;
        cin >> aristas;
        cin.ignore();

        //ERROR TAMAÑO LISTA

        if (num_nodos < 0 || num_nodos > MAX_NODOS) {
            cerr << "Numero de nodos (" << num_nodos << ") no
valido\n";
            exit(0);
        }

        GrafoLista lectura(num_nodos);

        //LECTURA ARISTAS
        for (int i= 0; i < aristas ; i++)
        {
            int v, w, veces;
            cin >> v >> w >> veces;
            cin.ignore();
            v--;
            w--; //AJUSTAR A INDICES ARRAY

            //INSERCION NODOS
            if(w >= 0 && w < num_nodos && v >= 0 && v < num_nodos )
```

```
            {
                lectura.insertarnodo(v,w,veces);
            }
        }
        return lectura;
    }

    int algoritmo_Dijkstra_coste_camino(int A)
    {
        vector<int> nodos_paso(num_nodos, A);
        vector<int> coste_caminos(num_nodos, INFINITO);
        vector<bool> nodos_escogidos(num_nodos, false);
        //INICIALIZACION
        nodos_escogidos[A] = true;
        coste_caminos[A] = 0;
        nodos_paso[A] = -1;
        NodoPtr* aux = listaAdyacencia[A];
        while ( aux != nullptr )
        {
            int v = aux -> w ;
            coste_caminos[v] = menor_valor_vector( aux -> datos );
            aux = aux -> sig;
        }

        //CUERPO
        for ( int i = 1; i < num_nodos; i ++)
        {
            //OBTENER EL NODO Y VALOR DEL CAMINO MAS CORTO
            ID id = elegido(nodos_escogidos, coste_caminos);

            //VERIFICAR SI QUEDAN NODOS ACCESIBLES
            if ( id.v == -1 )
            {
                break;
            }

            nodos_escogidos[id.v] = true;

            //AJUSTAR VALORES ADYACENTES A V
            actualizar(id, coste_caminos, nodos_paso);
        }

        return mayor_valor_vector(coste_caminos);
```

```
    }
};

int main (void)
{
    int num_casos;
    cin >> num_casos;
    cin.ignore();
    for (int i= 0; i < num_casos; i++)
    {
        GrafoLista cotilleo = GrafoLista::leeGrafo();
        int tiempo = cotilleo.algoritmo_Dijkstra_coste_camino(0);
        cout << tiempo << endl;
    }
  return 0;
}
```

407.

Analisis, diseño y eficiencia.

Para la resolución de este problema he declarado la clase GrafoLista para modular el código. Los atributos de esta clase son la lista de adyacencia implementada mediante la idea de listas ligadas, el vector visitados necesario para la bpp o la bpa(aunque en este caso no se usa) y el número de nodos que tendrá el grafo. La idea que he seguido para solucionar este problema ha sido aplicar el algoritmo de Dijkstra sobre el grafo que nos proporcionan, pero en vez de sobre un nodo solo, aplicarlo sobre todos los nodos. He decidido hacerlo así porque la ciudad que buscamos es la que tiene menor excentricidad, entonces aplicando el algoritmo tomando los diferentes nodos como el nodo de inicio obtenemos el par (ciudad, excentricidad), en el código

es la estructura candidatos. Luego comparamos las excentricidades de las distintas ciudades quedándonos con la menor para obtener la capital que buscamos.

En cuanto a la eficiencia, el algoritmo de Dijkstra tiene normalmente O(n*a*log(n)), siendo n el número de nodos y a la longitud media de las listas, pero cómo se aplica n veces, se tiene O(n^2*a*log(n)). Sin embargo; esta eficiencia se ve afectada por la función leeGrafo, ya que las inserciones en las listas se realizan al final(O(a), donde a es la longitud de la lista).

Listado de código.

```
#include <stdlib.h>
#include <string.h>
#include <vector>
#include <iostream>
using namespace std;

#define MAX_NODOS 200
#define INFINITO  2147483000

class GrafoLista
{
    private:
    struct NodoPtr
    {
        int w;
        int coste;
        NodoPtr* sig;
        NodoPtr():w(-1),sig(nullptr){};
        NodoPtr(int w, int coste): w(w), coste(coste), sig(nullptr){};;

    };
    struct ID   // para saber a que ciudad corresponde cada nodo
    {
        string ciudad;
        NodoPtr* adyacentes;
        ID(): ciudad(""), adyacentes(nullptr){};
        ID(string ciudad): ciudad(ciudad), adyacentes(nullptr){};
    };
    struct candidatos   //obtener los pares ciudad candidata a capital
y su excentridad desde cualquier nodo del grafo
    {
        string ciudad;
```

```
        int excentridad;
        candidatos(): ciudad(""), excentridad(-1){};
        candidatos( string ciudad, int excentridad): ciudad(ciudad),
excentridad(excentridad){};
        bool operator<(const candidatos& cand1) const{
            if ( excentridad != cand1.excentridad )
            {
                return excentridad < cand1.excentridad;
            }
            return ciudad < cand1.ciudad;
        }
    };
    int num_nodos;
    vector<bool> visitados;
    vector<ID> listaAdyacencia;

    void insertarnodo(int v, int w, int coste)
    {
        NodoPtr* insertar = new NodoPtr(w, coste);
        if ( this -> listaAdyacencia[v].adyacentes == nullptr)
        {
            this -> listaAdyacencia[v].adyacentes = insertar;
        }
        else
        {

            NodoPtr* aux = this -> listaAdyacencia[v].adyacentes;
            while ( aux -> sig != nullptr )
            {
                aux = aux -> sig;
            }
            aux -> sig = insertar;
        }
    }
    int elegido( vector<bool>& nodos_escogidos, vector<int>&
coste_caminos)
    {
        int minimo = INFINITO;
        int indice = -1;
        for ( int v = 0; v < num_nodos; v++)
        {
            if ( !nodos_escogidos[v] && coste_caminos[v] < minimo)
            {
```

```
                minimo = coste_caminos[v];
                indice = v;
            }
        }
        return indice;
    }
    void leerCiudades()
    {
        cin.ignore();
        for ( int i = 0; i < num_nodos; i++ )
        {
            getline(cin, listaAdyacencia[i].ciudad);
        }
    }
    int mayor_valor_vector( vector<int>& lista_costes)
    {
        int maximo = -1;
        for ( size_t i = 0; i < lista_costes.size(); i++)
        {
            if ( lista_costes[i] > maximo)
            {
                maximo = lista_costes[i];
            }
        }
        return maximo;
    }

    public:
    GrafoLista(): num_nodos(0){};
    GrafoLista(int num_nodos)
    {
        this->num_nodos = num_nodos;
        listaAdyacencia.assign(num_nodos, ID());
        visitados.assign(num_nodos, false);
    }

     static GrafoLista leeGrafo ()
    {
        //PARAMETROS

        int num_nodos;
        cin >> num_nodos;
```

```
        //ERROR TAMAÑO LISTA

        if (num_nodos < 0 || num_nodos > MAX_NODOS) {
            cerr << "Numero de nodos (" << num_nodos << ") no
valido\n";
            exit(0);
        }

        GrafoLista lectura(num_nodos);
        lectura.leerCiudades();

        //LECTURA ARISTAS
        while ( true )
        {
            int v, w, coste;
            cin >> v >> w >> coste;

            //VER SI PARAR LECTURA DE GRAFO
            if ( v == 0 && w == 0 && coste == 0 )
            {
                break;
            }

            //INSERCION NODOS
            if(w >= 0 && w < num_nodos && v >= 0 && v < num_nodos )
            {
                lectura.insertarnodo(v,w,coste);
                lectura.insertarnodo(w,v,coste);
            }
        }
        return lectura;
    }
    candidatos algoritmo_Dijkstra_coste_camino(int A)
    {
        vector<int> nodos_paso(num_nodos, A);
        vector<int> coste_caminos(num_nodos, INFINITO);
        vector<bool> nodos_escogidos(num_nodos, false);

        //INICIALIZACION
        nodos_escogidos[A] = true;
        coste_caminos[A] = 0;
        nodos_paso[A] = -1;
        NodoPtr* aux = listaAdyacencia[A].adyacentes;
```

```
        while ( aux != nullptr )
        {
            int v = aux -> w ;
            coste_caminos[v] = aux -> coste;
            aux = aux -> sig;
        }

        //CUERPO
        for ( int i = 1; i < num_nodos; i++ )
        {
            int v = elegido(nodos_escogidos, coste_caminos);
            if ( v == -1 )
            {
                break;
            }
            nodos_escogidos[v] = true;
            NodoPtr* aux = listaAdyacencia[v].adyacentes;
            while( aux != nullptr )
            {
                int w = aux ->w;
                int coste_v_w = coste_caminos[v] + aux ->coste;
                if ( !nodos_escogidos[w] && (coste_v_w <
coste_caminos[w]))
                {
                    coste_caminos[w] = coste_v_w;
                    nodos_paso[w] = v;
                }
                aux = aux -> sig;
            }
        }
        int excentridad = mayor_valor_vector( coste_caminos );
        return candidatos(listaAdyacencia[A].ciudad, excentridad);
    }
    void mostrar_capital()
    {
        candidatos capital = algoritmo_Dijkstra_coste_camino(0);
        for( int v = 1; v < num_nodos; v++ )
        {
            candidatos posible_capital =
algoritmo_Dijkstra_coste_camino(v);
            if ( posible_capital < capital )
            {
                capital = posible_capital;
```

```
            }
        }
        cout << capital.ciudad << endl << capital.excentridad << endl;
    }

};
int main (void)
{
    int num_casos;
    cin >> num_casos;
    cin.ignore();
    for (int i= 0; i < num_casos; i++)
    {
        GrafoLista caminos = GrafoLista::leeGrafo();
        caminos.mostrar_capital();
    }
  return 0;
}
```

408.

Analisis, diseño y eficiencia.

Para la resolución de este problema he declarado la clase GrafoLista para modular el código. Los atributos de esta clase son la lista de adyacencia implementada mediante la idea de listas ligadas, el vector visitados necesario para la bpp o la bpa, el número de nodos que tendrá el grafo y los casos por grafo. La idea que he seguido para solucionar este problema ha sido aplicar una búsqueda primero en anchura. Antes de comentar la idea principal, cabe destacar, que en la función leeGrafo primero se obtienen las palabras y se guardan en un vector auxiliar para poder ordenarlas alfabéticamente y así que la bpa se realice en orden alfabético. Otro detalle importante es la creación de las aristas entre nodos, ya que en este problema no se proporcionan directamente. Las aristas del grafo indican que dos palabras tienen el mismo tamaño y que difieren en una letra, ya que el problema solo permite avanzar a una palabra que tenga una letra diferente a la anterior. Una vez hechas estas observaciones, las modificaciones que

tiene mi bpa respecto al normal: un vector en el que se va almacenando las palabras que se van recorriendo hasta llegar a la palabra deseada y un booleano para indicar si ya hemos llegado a la palabra deseada. Para guardar el recorrido se usa la estrategia de punteros al padre, ya que la bpa, a diferencia de la bpp, va visitando el grafo por capas entonces no se pueden ir insertando directamente los nodos a los que se acceden, sino que se debe ir guardando el nodo desde que se accedió al actual. Finalmente, la búsqueda devuelve un vector vacío si no se puede construir el camino o un vector con el camino correspondiente. Estos casos se diferencian en el main y se imprime el texto correspondiente al caso que se esté tratando.

En cuanto a la eficiencia, con la búsqueda primero en anchura tendremos O(n+a), siendo n el número de nodos y a la longitud media de las listas. Sin embargo; esta eficiencia se ve afectada por la función leeGrafo, ya que las inserciones en las listas se realizan al final(O(a), donde a es la longitud de la lista).

Listado de código.

```
#include <stdlib.h>
#include <string.h>
#include <vector>
#include <queue>
#include <algorithm>
#include <iostream>
using namespace std;

#define MAX_NODOS 17000
#define MAX_LONG 40
#define MIN_LONG 4

class GrafoLista
{
    private:
    struct ID   // para saber a que palabra corresponde cada nodo
    {
        string palabra;
        vector<int> adyacentes;
        ID(): palabra(""), adyacentes(){};
        ID(string palabra): palabra(palabra), adyacentes(){};
    };
    int num_nodos;
    int casos;
    vector<bool> visitados;
    vector<ID> listaAdyacencia;
```

```
        bool isArista( string palabra1, string palabra2 ) //funcion
para ver si hay arista entre dos palabras
        {
            int contador_letras_diferentes = 0;
            size_t letra = 0;
            if ( palabra1.size() == palabra2.size() )
            {
                while ( letra < palabra1.size() &&
contador_letras_diferentes < 2 )
                {
                    if ( palabra1[letra] != palabra2[letra] )
                    {
                        contador_letras_diferentes++;
                    }
                    letra++;
                }
                if ( contador_letras_diferentes > 1 )
                {
                    return false;
                }
                return true;
            }
            return false;

        }
        int buscar_palabra ( string palabra ) //se supone q i nunca
sera igual o  mayor a num_nodos por ser una palabra que sabemos que
esta
        {
            int i = 0;
            while ( listaAdyacencia[i].palabra != palabra )
            {
                i++;
            }
            return i;
        }

        void copiar_palabras_a_grafo( vector<string>& lista_palabras
)
        {
            for ( int v = 0; v < num_nodos; v++)
            {
```

```
                    listaAdyacencia[v].palabra = lista_palabras[v];
                }
            }
            void leer_palabras( vector<string>& palabras)
            {
                string pal;
                for ( int v = 0; v < num_nodos; v++)
                {
                    cin >> pal;
                    int longitud = pal.size();
                    if (longitud < MIN_LONG || longitud > MAX_LONG)
                    {
                        cerr << "Longitud de palabra (" << pal << ") no
valida\n";
                        exit(0);
                    }
                    palabras[v] = pal;
                }
                sort( palabras.begin(),  palabras.end());
            }

            void insertar_nodos( )
            {
                for ( int v = 0; v < num_nodos; v++)
                {
                    for( int w = v +1; w < num_nodos; w++)
                    {
                        if ( isArista ( listaAdyacencia[v].palabra,
listaAdyacencia[w].palabra ))
                        {
                            listaAdyacencia[v].adyacentes.push_back(w);
                            listaAdyacencia[w].adyacentes.push_back(v);
                        }
                    }
                }
            }

            public:
            GrafoLista(): num_nodos(0){};
            GrafoLista(int num_nodos, int casos)
            {
                this->num_nodos = num_nodos;
                this->casos = casos;
```

```
            listaAdyacencia.assign(num_nodos, ID());
            visitados.assign(num_nodos, false);
        }

         static GrafoLista leeGrafo ()
        {
            //PARAMETROS

            int num_nodos, casos;
            cin >> num_nodos >> casos;

            //ERROR TAMAÑO LISTA

            if (num_nodos < 0 || num_nodos > MAX_NODOS) {
                cerr << "Numero de nodos (" << num_nodos << ") no
valido\n";
                exit(0);
            }

            GrafoLista lectura(num_nodos, casos);
            vector<string> palabras_ordenadas(num_nodos); //se usa
una lista intermedia para poder guardar las palabras ordenadas en el
grafo
            lectura.leer_palabras( palabras_ordenadas ); //guarda las
palabras en un vector y los inserta en orden alfabetico para luego
cuando haga bpa se obtenga el resultado querido
            lectura.copiar_palabras_a_grafo( palabras_ordenadas); //
insertar la lista ordenada al grafo
            lectura.insertar_nodos(); //establece los nodos del grafo
            return lectura;
        }
        int getCasos()
        {
            return casos;
        }
        int getPosicion( string palabra )
        {
            int i = 0;
            while ( i < num_nodos && listaAdyacencia[i].palabra !=
palabra)
            {
                i++;
            }
```

```
            return i;
        }
        void bpa(int v, int w, vector<int>& recorrido, bool&
encontrado)
        {
            queue<int> cola;
            visitados[v] = true;
            cola.push(v);
            while ( !cola.empty())
            {
                int x = cola.front();
                cola.pop();
                if ( x == w )
                {
                    encontrado = true;
                    break;
                }
                for ( size_t i = 0; i <
listaAdyacencia[x].adyacentes.size(); i++)
                {
                    int w = listaAdyacencia[x].adyacentes[i];
                    if( !visitados[ w ] )
                    {
                        visitados [ w ]= true;
                        recorrido[ w ] = x;
                        cola.push( w );
                    }
                }
            }


        }
        vector<string> busquedaPA( int v, int w ) //usa la estrategia
de punteros al padre para guardar los recorridos que produce bpa
        {
            visitados.assign(num_nodos, false);
            vector<int> recorrido(num_nodos, -1);
            bool encontrado = false;
            bpa(v, w, recorrido, encontrado);
            if ( !encontrado )
            {
                return vector<string>();
            }
```

```
            vector<string> camino_palabras;
            int i = w;
            while ( i != -1 )    //obtener secuencia de recorrido
            {
                camino_palabras.push_back(
listaAdyacencia[i].palabra);
                i = recorrido[i];
            }
            return camino_palabras; //el vector esta con el orden
contrario
        }


    };
    int main (void)
    {
        GrafoLista diccionario = GrafoLista::leeGrafo();
        int num_casos = diccionario.getCasos();
        vector<string> resultado;
        string pal_inicial, pal_final;
        for (int i= 0; i < num_casos; i++)
        {
            cin >> pal_inicial >> pal_final;
            int pos_palabra_ini =
diccionario.getPosicion(pal_inicial);
            int pos_palabra_fin = diccionario.getPosicion(pal_final);
            resultado = diccionario.busquedaPA( pos_palabra_ini,
pos_palabra_fin);
            cout << "Caso " << i + 1 << endl ;
            if ( resultado.empty() )
            {
                cout << "NO HAY SOLUCION" << endl;
            }
            else
            {
                cout << resultado.size() << endl;
                for ( size_t j = resultado.size(); j > 0; j--)
//porque el vector esta en orden inverso
                {
                    cout << resultado[j - 1] << endl;
                }
            }
```

```
        }
      return 0;
    }
```

409.

Analisis, diseño y eficiencia.

Para la resolución de este problema he declarado la clase GrafoLista para modular el código. Los atributos de esta clase son la lista de adyacencia implementada mediante la idea de listas ligadas, el vector visitados necesario para la bpp o la bpa y el número de nodos que tendrá el grafo. La idea que he seguido para solucionar este problema ha sido aplicar el algoritmo de R.Tarjan de los apuntes. He empleado este método porque nos dice que una página es la misma que otra si podemos llegar a la segunda desde la primera mediante un camino, es decir, si ambas páginas están en el mismo componente conexo. Entonces eso hace que el problema se convierta en un problema de ver los diferentes componentes conexos que tiene el grafo que es exactamente lo que realiza el algoritmo de R.Tarjan. Para llevarlo a

cabo primero se realiza una bpp que guarda el recorrido en un vector. Luego habría que recorrer un grafo invertido al que tenemos y en orden contrario al del primer bpp. Para crear el grafo invertido he añadido un parámetro extra a la función leeGrafo que modifique un grafo para que sea el grafo inverso del que devuelve la propia función. Una vez que tenemos el grafo necesario para realizar la segunda bpp, usamos el vector devuelto por la primera bpp, pero lo recorremos en orden inverso ya que es el camino que debemos seguir. Esta última bpp devuelve una matriz bidimensional en la que cada fila se van almacenando las distintas componentes conexas. Otra cosa a destacar es el uso de sort para ordenar los vectores, ya que la salida debe estar ordenada. Finalmente, el main consta únicamente de hacer un recorrido por la matriz bidimensional para imprimir los resultados.

En cuanto a la eficiencia, con el algoritmo R.Tarjan tendremos O(2(n+a))=O(n+a), siendo n el número de nodos y a la longitud media de las listas. Sin embargo; esta eficiencia se ve afectada por la función leeGrafo, ya que las inserciones en las listas se realizan al final(O(a), donde a es la longitud de la lista) y por las operaciones sort.

Listado de código.

```
#include <stdlib.h>
#include <string.h>
#include <vector>
#include <iostream>
#include <sstream>
#include <algorithm>
using namespace std;

class GrafoLista
{
    private:
    struct NodoPtr
    {
        int w;
        NodoPtr* sig;
        NodoPtr():w(-1),sig(nullptr){};
        NodoPtr( int w ): w(w), sig(nullptr){};;

    };
    int num_nodos;
    vector<bool> visitados;
    vector<NodoPtr*> listaAdyacencia;

    void insertarnodo(int v, int w) //funcion auxiliar para
insertar (v,w)
```

```
        {
            NodoPtr* insertar = new NodoPtr(w);
            if ( this -> listaAdyacencia[v] == nullptr)
            {
                this -> listaAdyacencia[v] = insertar;
            }
            else
            {

                NodoPtr* aux = this -> listaAdyacencia[v];
                while ( aux -> sig != nullptr )
                {
                    aux = aux -> sig;
                }
                aux -> sig = insertar;
            }
        }

        public:
        GrafoLista(): num_nodos(0){};

        GrafoLista(int num_nodos)
        {
            this->num_nodos = num_nodos;
            listaAdyacencia.assign(num_nodos, nullptr);
            visitados.assign(num_nodos, false);
        }

        static GrafoLista leeGrafo ( GrafoLista& invertido, int
num_nodos) //crea el grafo normal y modifica el pasado como parametro
para que sea el invertido
        {
//del que nos proporcionan para poder aplicar el algoritmo de R.Tarjan
            GrafoLista normal(num_nodos);
            //LECTURA ARISTAS
            for (int v= 0; v < num_nodos ; v++)
            {
                int w;
                cin >> w;
                while ( w != 0)
                {
                    normal.insertarnodo(v,w-1);
                    invertido.insertarnodo(w-1,v);
```

```
                    cin >> w;
                }
            }
            return normal;
        }
        void bpp(int v, vector<int>& recorrido) //bpp normal que va
guardando el recorrido en el vector recorrido
        {
            visitados[v] = true;
            NodoPtr* aux = listaAdyacencia[v];
            while ( aux != nullptr )
            {
                if ( !visitados[ aux -> w ])
                {
                    bpp( aux -> w, recorrido);
                }
                aux = aux -> sig;
            }
            recorrido.push_back(v);
        }
        vector<int> busquedaPP_normal() //devuelve el vector de
recorrido que luego habra que recorrerlo en orden contrario
        {
            visitados.assign(num_nodos, false);
            vector<int> recorrido;
            for ( int v = 0; v < num_nodos; v++)
            {
                if ( !visitados[v] )
                {
                    bpp(v, recorrido);
                }
            }
            return recorrido;
        }
        vector<vector<int>> busquedaPP_invertido( vector<int>&
recorrido , int& filas ) // devuelve una matriz bidimensional donde las
filas son los diferentes componentes conexos
        {
            int nodos_visitados = 0;
            visitados.assign(num_nodos, false);
            vector<vector<int>> componentes_fuertemente_conexos;
            for ( int i = num_nodos - 1; i >= 0; i-- )
            {
```

```
                int v = recorrido[i];
                if ( !visitados[v] )
                {
                    vector<int> componente_conexa;
                    bpp( v, componente_conexa);
                    sort(componente_conexa.begin(),
componente_conexa.end()); //ordeno porque la solucion necesita que se
ordenen los nodos que forman la componente conexa
                    nodos_visitados += componente_conexa.size();
                    componentes_fuertemente_conexos.push_back(
componente_conexa );
                    filas++;
                }
            }
            return componentes_fuertemente_conexos;
        }

    };

    int main (void)
    {
        int num_casos, num_nodos;
        cin >> num_casos;
        for (int i= 0; i < num_casos; i++)
        {
            cin >> num_nodos;
            int filas = 0;
            GrafoLista invertido(num_nodos);
            GrafoLista normal = GrafoLista::leeGrafo(invertido,
num_nodos);
            vector<int> recorrido = normal.busquedaPP_normal();
            vector<vector<int>> componentes_conexas =
invertido.busquedaPP_invertido(recorrido, filas);
            sort(componentes_conexas.begin(),
componentes_conexas.end()); //porque los filas tambien tienen que estar
ordenadas
            cout << "Caso " << i+1 << ": total " << filas << endl;
            for ( int j = 0; j < filas; j++)
            {
                cout << "Simple " << j+1 << ":";
                size_t v = 0;
                while( v < componentes_conexas[j].size() )
                {
```

```
                    cout << " " << componentes_conexas[j][v] + 1;
                    v++;
                }
                cout << endl;
            }
        }
    return 0;
}
```

411.

Analisis, diseño y eficiencia.

Para la resolución de este problema he declarado la clase GrafoLista para modular el código. Los atributos de esta clase son la lista de adyacencia implementada mediante la idea de listas ligadas, el vector visitados necesario para la bpp o la bpa y el número de nodos que tendrá el grafo. La idea que he seguido para solucionar este problema ha sido aplicar una búsqueda primero en profundidad. Aunque es un problema de ver qué nodos están en que componente conexa, he optado por realizar una bpp, ya que en la función principal de esta busqueda (busquedaPP) tiene un bucle for que sirve para visitar la siguiente componente conexa del grafo para así poder recorrer todo el grafo. He aprovechado esto y he usado la variable clase_equivalencia para llevar un control del número de islas que hay, las cuales irán creciendo si se vuelve a acceder al if del bucle porque eso implica que quedan nodos que visitar que no estaban en la componente conexa anterior. Esta

variable también nos ayuda para llevar el control de qué nodos pertenecen a qué isla. Por lo demás es una bpp normal que devuelve un vector donde el contenido de este es la clase de equivalencia(isla) a la que pertenecen cada nodo. Finalmente, el main es solo imprimir los resultados que contiene el vector obtenido por la búsqueda.

En cuanto a la eficiencia, con la búsqueda primero en profundidad tendremos O(n+a), siendo n el número de nodos y a la longitud media de las listas. Sin embargo; esta eficiencia se ve afectada por la función leeGrafo, ya que las inserciones en las listas se realizan al final(O(a), donde a es la longitud de la lista.

Listado de código.

```
#include <stdlib.h>
#include <string.h>
#include <vector>
#include <iostream>
#include <sstream>
using namespace std;

class GrafoLista
{
    private:
    struct NodoPtr
    {
        int w;
        NodoPtr* sig;
        NodoPtr():w(-1),sig(nullptr){};
        NodoPtr( int w ): w(w), sig(nullptr){};;

    };
    int num_nodos;
    vector<bool> visitados;
    vector<NodoPtr*> listaAdyacencia;

    void insertarnodo(int v, int w)
    {
        NodoPtr* insertar = new NodoPtr(w);
        if ( this -> listaAdyacencia[v] == nullptr)
        {
            this -> listaAdyacencia[v] = insertar;
        }
        else
```

```
        {

            NodoPtr* aux = this -> listaAdyacencia[v];
            while ( aux -> sig != nullptr )
            {
                aux = aux -> sig;
            }
            aux -> sig = insertar;
        }
    }
    public:
    GrafoLista(): num_nodos(0){};

    GrafoLista(int num_nodos)
    {
        this->num_nodos = num_nodos;
        listaAdyacencia.assign(num_nodos, nullptr);
        visitados.assign(num_nodos, false);
    }

     static GrafoLista leeGrafo ()
    {
        //PARAMETROS

        int num_nodos;
        int aristas;
        cin >> num_nodos;
        cin >> aristas;

        GrafoLista lectura(num_nodos);

        //LECTURA ARISTAS
        for (int i= 0; i < aristas ; i++)
        {
            int v, w;
            cin >> v >> w;

            v--;
            w--; //AJUSTAR A INDICES ARRAY

            //INSERCION NODOS
            if(w >= 0 && w < num_nodos && v >= 0 && v < num_nodos )
            {
```

```
                lectura.insertarnodo(v,w);
                lectura.insertarnodo(w,v);
            }
        }
        return lectura;
    }
    void bpp(int v, vector<int>& relaciones, int clase_equivalencia)
    {
        visitados[v] = true;
        relaciones[v] = clase_equivalencia;
        NodoPtr* aux = listaAdyacencia[v];
        while ( aux != nullptr )
        {
            if ( !visitados[ aux -> w ])
            {
                bpp( aux -> w, relaciones, clase_equivalencia);
            }
            aux = aux -> sig;
        }
    }
    vector<int> busquedaPP(int& NI)
    {
        visitados.assign(num_nodos, false);
        vector<int> relaciones(num_nodos, -1);
        int clase_equivalencia = 1;
        for ( int v = 0; v < num_nodos; v++)
        {
            if ( !visitados[v] )
            {
                bpp(v, relaciones, clase_equivalencia);
                clase_equivalencia++;
            }
        }
        NI = clase_equivalencia - 1;
        return relaciones;
    }
};

int main (void)
{
    int num_casos;
    cin >> num_casos;
    vector<int> relaciones;
```

```
    int NI = 0;
    for (int i= 0; i < num_casos; i++)
    {
        GrafoLista islas = GrafoLista::leeGrafo();
        relaciones = islas.busquedaPP(NI);
        cout << "Caso " << i+1 << endl << NI << endl;
        for ( size_t v = 0; v < relaciones.size(); v++)
        {
            cout << relaciones[v] << endl;
        }
    }
  return 0;
}
```

412.

Analisis, diseño y eficiencia.

Para la resolución de este problema he declarado la clase GrafoLista para modular el código. Los atributos de esta clase son la lista de adyacencia implementada mediante la idea de listas ligadas, el vector visitados necesario para la bpp o la bpa y el número de nodos que tendrá el grafo. La idea que he seguido para solucionar este problema ha sido aplicar el algoritmo de Prim. He usado este algoritmo porque es un problema de caminos mínimos, pero lo que importa no es el mínimo global del recorrido sino el mínimo coste de aristas. Esto se debe a que debemos ver el mayor coste del viaje de un planeta a otro para que nuestro tanque de energía tenga ese valor. Para llevar a cabo el algoritmo, he implementado la idea vista en los apuntes. Este algoritmo devuelve el grafo al cual se le han eliminado las aristas innecesarias. Para crear el grafo se usa la función transformar_Grafo que se basa en los vectores que se usan de apoyo en el algoritmo

principal. Una vez tenemos el grafo modificado realizamos una bpp sobre él para obtener el mayor coste entre nodos. Usamos para ello un booleano para indicar si se ha podido llegar al planeta final desde el inicial y dos ints para aprovechar la recursión y obtener el coste que buscamos. Finalmente, el main consta de devolver el coste conseguido en la bpp.

En cuanto a la eficiencia, con el algoritmo de Prim tendremos O(n^2), siendo n el número de nodos. Sin embargo; esta eficiencia se ve afectada por la función leeGrafo, ya que las inserciones en las listas se realizan al final(O(a), donde a es la longitud de la lista) y por la bpp (O(n+a)).

Listado de código.

```
#include <stdlib.h>
#include <string.h>
#include <queue>
#include <vector>
#include <iostream>
#include <sstream>
using namespace std;

#define INFINITO  2147483000
class GrafoLista
{
    private:
    struct NodoPtr
    {
        int w;
        int coste;
        NodoPtr* sig;
        NodoPtr():w(-1),sig(nullptr){};
        NodoPtr( int w, int coste ): w(w), coste(coste),
sig(nullptr){};;

    };
    int num_nodos;
    vector<bool> visitados;
    vector<NodoPtr*> listaAdyacencia;

    void insertarnodo(int v, int w, int coste)
    {
        NodoPtr* insertar = new NodoPtr(w, coste);
        if ( this -> listaAdyacencia[v] == nullptr)
```

```
        {
            this -> listaAdyacencia[v] = insertar;
        }
        else
        {

            NodoPtr* aux = this -> listaAdyacencia[v];
            while ( aux -> sig != nullptr )
            {
                aux = aux -> sig;
            }
            aux -> sig = insertar;
        }
    }
    int indice_menor_valor_vector( vector<int>& lista_costes)
    {
        int minimo = INFINITO;
        int v = -1;
        for ( size_t i = 0; i < lista_costes.size(); i++)
        {
            if ( !visitados[i] && lista_costes[i] < minimo)
            {
                minimo = lista_costes[i];
                v = i;
            }
        }
        if( v != -1 )
        {
            visitados[v] = true;;
        }
        return v;
    }
    GrafoLista transformar_Grafo( vector<int>& mas_cercano,
vector<int>& coste ) // interpretar el resultado del algoritmo de prim
para hacer el grafo querido
    {
        GrafoLista transformacion(num_nodos);
        for( int v = 0; v < num_nodos; v++)
        {
            int w = mas_cercano[v];
            if ( w != -1 )
            {
                transformacion.insertarnodo(v, w, coste[v] );
```

```
                transformacion.insertarnodo(w, v, coste[v] );
            }
        }
        return transformacion;
    }
    int mayor( int x, int y)
    {
        if ( x < y)
        {
            return y;
        }
        return x;
    }
    void bpp(int v, int final, bool& encontrado, int max_actual, int&
resultado)
    {
        visitados[v] = true;
        if ( final == v )
        {
            encontrado = true;
            resultado = max_actual;
            return;
        }

        NodoPtr* aux = listaAdyacencia[v];
        while ( aux != nullptr )
        {
            if ( !visitados[ aux -> w ])
            {
                bpp( aux -> w, final, encontrado, mayor( max_actual,
aux -> coste), resultado);
                if ( encontrado == true )
                {
                    return;
                }
            }
            aux = aux -> sig;
        }
    }

    public:
    GrafoLista(): num_nodos(0){};
    GrafoLista(int num_nodos)
```

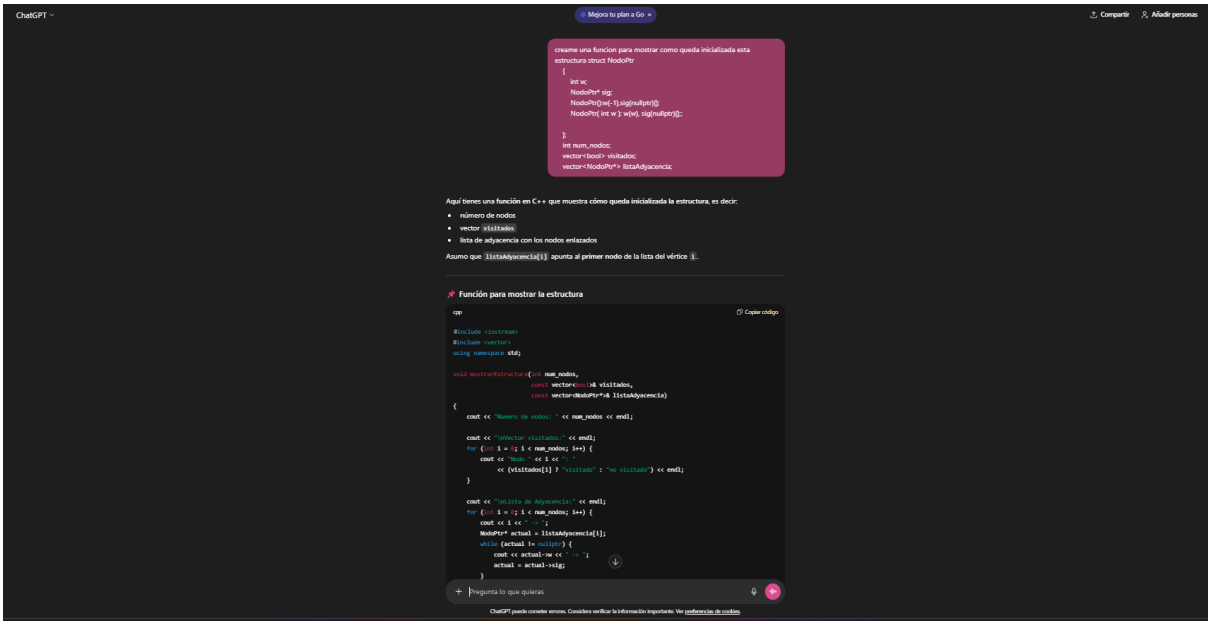
```
    {
        this->num_nodos = num_nodos;
        listaAdyacencia.assign(num_nodos, nullptr);
        visitados.assign(num_nodos, false);
    }
     static GrafoLista crearGrafo ( int num_nodos, int aristas)
    {
        GrafoLista lectura(num_nodos);

        //LECTURA ARISTAS
        for (int i= 0; i < aristas ; i++)
        {
            int v, w, coste;
            cin >> v >> w >> coste;

            //INSERCION NODOS
            if(w >= 0 && w < num_nodos && v >= 0 && v < num_nodos )
            {
                lectura.insertarnodo(v,w,coste);
                lectura.insertarnodo(w,v,coste);
            }
        }
        return lectura;
    }
    GrafoLista algoritmo_Prim(int inicio)
    {
        vector<int> menor_coste(num_nodos, INFINITO);
        vector<int> mas_cercano(num_nodos, -1);
        int v;
        visitados.assign(num_nodos, false);

        //inicializacion
        visitados[inicio] = true;
        NodoPtr* aux = listaAdyacencia[inicio];
        while ( aux != nullptr )
        {
            mas_cercano[ aux -> w ] = inicio;
            menor_coste[ aux -> w ] = aux -> coste;
            aux = aux -> sig;
        }

        //algoritmo Prim
       for( int i = 1; i <  num_nodos; i++)
```

```
        {
            v = indice_menor_valor_vector(menor_coste);
            if( v == -1)
            {
                break;
            }

            //actualizacion
            aux = listaAdyacencia[v];
            while( aux != nullptr)
            {
                int valor_v_w = aux -> coste;
                int w = aux -> w;
                if ( valor_v_w < menor_coste[w] && !visitados[w] )
                {
                    menor_coste[w] = valor_v_w;
                    mas_cercano[w] = v;
                }
                aux = aux -> sig;
            }

        }
        return transformar_Grafo( mas_cercano, menor_coste);
    }
    int busquedaPP(int inicio, int final)
    {
        visitados.assign(num_nodos, false);
        int resultado = 0;
        bool encontrado = false;
        bpp(inicio, final, encontrado, 0, resultado);
        return resultado;
    }
};
int main (void)
{
    int num_nodos, aristas, casos, inicio, final;
    bool primero = true;
    cin >> num_nodos >> aristas;
    while (num_nodos != 0 && aristas != 0)
    {
        GrafoLista viaje = GrafoLista::crearGrafo(num_nodos, aristas);
        GrafoLista viaje_modificado = viaje.algoritmo_Prim(0);
        cin >> casos;
```

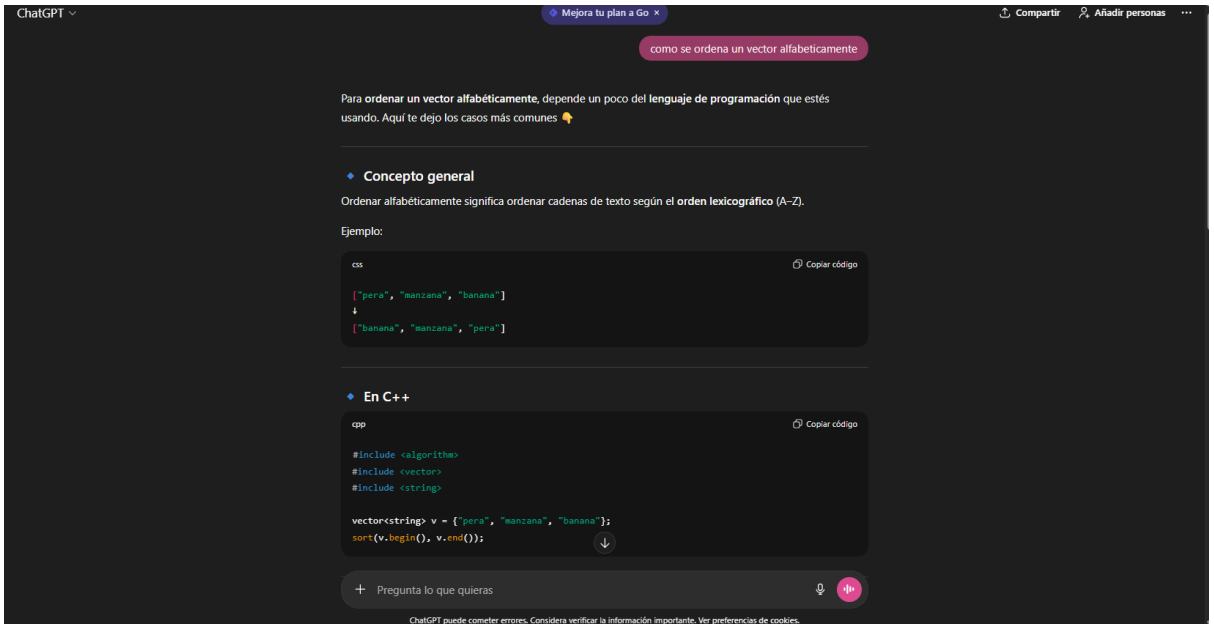
```
        if ( !primero )
        {
            cout << endl;
        }
        primero = false;

        for ( int i = 0; i < casos; i++)
        {
            cin >> inicio >> final;
            cout << viaje_modificado.busquedaPP(inicio, final) << endl;
        }
        cin >> num_nodos >> aristas;
    }
  return 0;
}
```

USO DE IA.

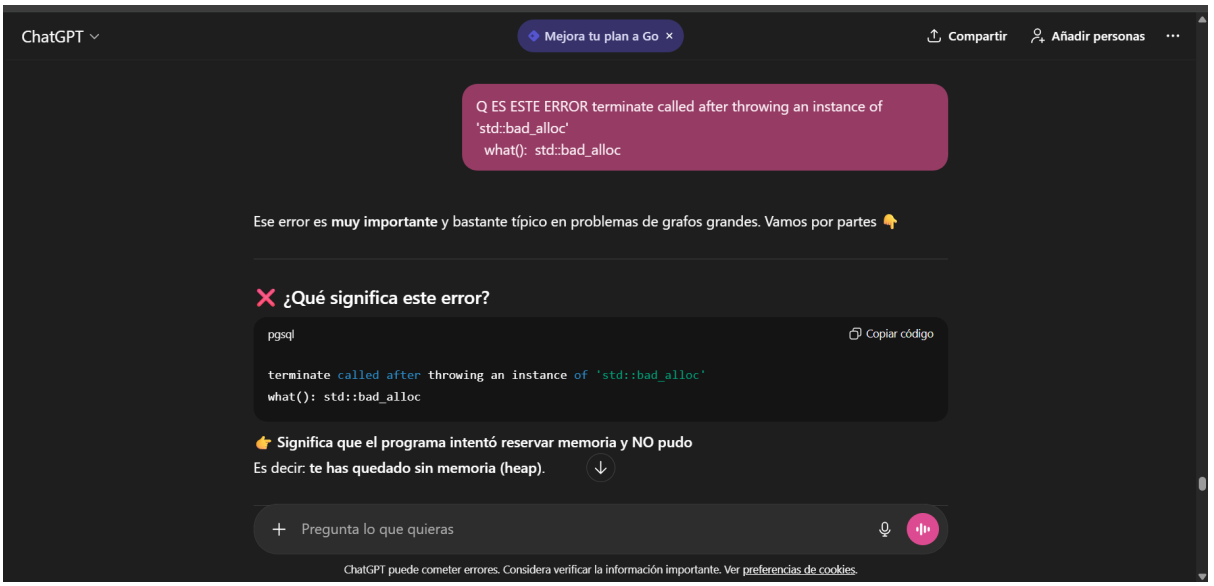
En cuanto a la Ia, el uso que he realizado de ella ha sido para que me creara funciones que mostraran el grafo o vectores y así poder ver como iba afectando en este cada actualización (inserción de nodos…) . Esto principalmente lo hice para optimizar el tiempo.



También me apoyé en ella para saber cómo se ordenaban los vectores para los ejercicios 408 y 409.



Por último, la use para saber que error me daba un programa (408). Este fallo era que se acababa la memoria virtual y por eso cambia la implementación de la lista de adyacencia de lista enlazada a un vector.


ChatGPT
Mejora tu plan a Go
Compartir
Añadir personas
Q ES ESTE ERROR terminate called after throwing an instance of 'std::bad_alloc'
what(): std::bad_alloc
Ese error es muy importante y bastante típico en problemas de grafos grandes. Vamos por partes 👇
❌ ¿Qué significa este error?
pgsql
Copiar código
terminate called after throwing an instance of 'std::bad_alloc'
what(): std::bad_alloc
👉 Significa que el programa intentó reservar memoria y NO pudo
Es decir: te has quedado sin memoria (heap).
Pregunta lo que quieras
ChatGPT puede cometer errores. Considera verificar la información importante. Ver preferencias de cookies.

CONCLUSIONES.

Al inicio del trabajo no entendía mucho como funcionaba cada algoritmo de los grafos y me costaba mucho identificar cual era necesario para un problema concreto. Por eso, sobre todo al principio, hice más uso de la función(la comentada anteriormente) proporcionada por la la para ver como iba quedando cada estructura en diferentes pasos de la ejecución. Esto me ayudó a visualizar los errores de planteamiento. Con el tiempo fui asimilando el funcionamiento y lógica de los algoritmos y ya tenía más soltura a la hora de plantear el problema. Por eso, considero que este tipo de trabajos ayudan más a afianzar los conocimientos, ya que para poder usar cada algoritmo o herramientas de teoría tienes que entender su comportamiento real y no solo como se escribe cada algoritmo.

Por otro lado, el tiempo empleado para llevar a cabo esta práctica ha sido alrededor de 12 horas.